



Gestion des secrets avec OpenStack Barbican

Conception et développement d'une plateforme web de gestion centralisée des secrets cryptographiques

AUTEURS

Rachel ANABA NGOUMOU
Harvis FOTSEU Landry

FORMATION

Master 2 Sécurité Cloud
Saint-Jean Ingénieur

EXAMINATEUR

Monsieur
NGUIMBUS Emmanuel

ANNÉE

2025
- 2026



Contexte et problématique

Notre projet, à savoir la gestion des secrets avec Barbican, est une réponse aux constats suivants :

1



Authentification Complexe

La complexité de l'authentification des applications tierces pour pouvoir accéder aux services de Barbican; seuls les utilisateurs ou les services OpenStack ont des comptes.

2



Renouvellement Manuel

Le renouvellement manuel des clés API et des certificats SSL/TLS; causant l'interruption des services qui en dépendent en cas d'expiration.

3



Absence de Notification

L'absence de notification pour signaler les changements de clés API ou de certificats aux applications et aux utilisateurs, ainsi que les potentiels erreurs.

PROBLÉMATIQUE CENTRALE

« Comment fournir aux développeurs un moyen simple et sécurisé de stocker, accéder et renouveler leurs secrets, sans jamais les exposer dans le code ? »





Objectifs du projet



Interface de Gestion Centralisée

Développer une interface web intuitive pour la gestion des secrets, parfaitement intégrée à l'écosystème **OpenStack Barbican**.



API REST Sécurisée

Permettre aux applications tierces de récupérer leurs secrets dynamiquement via une API REST sécurisée, éliminant les secrets codés en dur.



Renouvellement Automatisé

Automatiser le cycle de vie complet des certificats SSL/TLS et des **clés API** pour garantir une sécurité continue sans intervention manuelle.



Notifications et Audit

Notifier les propriétaires en temps réel pour toute action critique (création, accès, suppression) et assurer une traçabilité complète des accès.

Technologies utilisées

4

CATÉGORIE	TECHNOLOGIE	RÔLE
 Backend	Django 6 (Python 3.12)	Framework principal
 Frontend	Bootstrap 5 + HTMX	UI responsive, requêtes partielles sans rechargement
 Secrets	OpenStack Barbican 7.3	Stockage chiffré des secrets
 Identité	OpenStack Keystone	Authentification et isolation par application
 Crypto	pyOpenSSL, cryptography	Génération locale SSL/TLS et paires de clés RSA
 Email	Gmail SMTP	Notifications utilisateur (alertes de sécurité)
 API	Django REST Framework	Endpoints JSON publics pour applications tierces
 BDD	SQLite (dev)	Métadonnées applicatives (références uniquement)

 Stack technique basée sur des standards open-source éprouvés

Architecture Générale



Figure 1. Architecture Simplifiée

Flux Principal

- 1 Authentification**
L'utilisateur s'authentifie via Keystone.
- 2 Création d'Application**
Un projet Keystone Isolé est généré automatiquement.
- 3 Ajout de Secrets**
Stockés de manière chiffrée dans Barbican.
- 4 Génération de Jeton**
Création d'un token d'accès pour l'application tierce.
- 5 Lecture via API**
L'app tierce récupère le secret via l'API REST.

Composants de l'application



Gestion Utilisateur

`accounts`

Gère l'inscription, la connexion et le profil utilisateur. Centralise la modification des informations personnelles (photo, mot de passe) et l'interaction initiale avec Keystone pour la création du compte.



Gestion des Secrets

`secrets_manager`

Cœur fonctionnel de la plateforme. Permet l'ajout, le renouvellement et la suppression sécurisée.



Gestion des Applications

`apps_manager`

Responsable de la création et du cycle de vie des applications.

→ Point clé :

Chaque application correspond à un **projet Keystone isolé** possédant ses propres credentials Barbican, garantissant une étanchéité totale entre les environnements.



API & Accès Tiers

`api_manager`

Expose les secrets aux applications tierces via une API REST sécurisée. Gère la génération des jetons d'accès **Bearer** et assure la traçabilité des accès.

Architecture technique détaillée

7

Pile Technologique & Flux

FRONTEND / CLIENT



Navigateur Utilisateur

Interface Web + App Tierce

HTML5

Bootstrap 5

HTMX

BACKEND / DJANGO 6 (PYTHON 3.12)



accounts

Auth, Profil, Session



apps_manager

Gestion Apps & Projets



secrets_manager

Logique Secrets, Daemon



api_manager

REST API, Tokens, Mail

Keystone

Identité & Projets



Barbican

Stockage Chiffré



OpenSSL

PKI & Certs



SMTP

Notifications

Modèle de Données (Simplifié)

Utilisateur (Django)



username	varchar
email	varchar
keystone_user_id	UUID

↓ 1:N

Application



nom	varchar
keystone_project_id	UUID
app_cred_id	UUID

↓ 1:N

ConteneurSecret



nom	varchar
type	enum
ref_new_conteneur	URL
ref_old_conteneur	URL
date_expiration	datetime

↓ 1:N

↓ 0:1

Secret

ref_secret	URL
type	enum

TokenAcces

token_hash	sha256
created_at	date



Types de secrets supportés



Secret générique

type: generic

- ✓ Stockage de clés API, tokens d'accès ou chaînes de connexion base de données.
- ✓ Upload d'une clé API ou génération automatique.
- ✓ Stocké comme **texte chiffré** dans un conteneur Barbican.



Certificat SSL/TLS

type: certificate

- ✓ Upload d'un certificat existant (PEM) ou génération automatique via OpenSSL.
- ✓ Renouvellement automatique avec **archivage** de l'ancienne version.



Paire de clés RSA

type: rsa

- ✓ Upload d'une paire existante ou génération automatique (2048/4096 bits).
- ✓ Récupération séparée de la **clé publique** ou **privée** via l'API.

Architecture de Sécurité



Isolation par projet Keystone

Chaque application utilisateur possède son propre **projet Keystone dédié**.

Un utilisateur ne peut jamais accéder aux secrets d'un autre, même en cas de défaut applicatif (cloisonnement strict IAM).



Chiffrement côté Barbican

Les secrets ne sont **jamais stockés en clair** dans la base de données de BarbiLink.

Seules les références Barbican (URLs) sont conservées localement. Le payload chiffré reste dans le HSM/Backend sécurisé d'OpenStack.

Contrôles Opérationnels



Jetons à portée limitée

Chaque Jeton Bearer donne accès à **un seul secret**. La compromission d'un jeton n'expose pas les autres secrets de l'application.



Déconnexion automatique

Session expirée après **3 minutes d'inactivité** pour limiter les risques de vol de session (Hijacking) sur poste laissé ouvert.



Audit par email

Chaque accès API (lecture de secret) déclenche une **notification au propriétaire**, incluant l'adresse IP source de la requête.

Interface web — Écrans principaux

10



Dashboard

Vue d'ensemble synthétique : nombre d'applications et total des secrets. Graphiques de répartition par type.

Vue globale

Statistiques



Mes applications

Liste paginée de toutes les applications créées. Inclut une barre de recherche globale et des options de suppression individuelle ou en masse.

Liste paginée

Recherche



Détail d'une application

Liste des secrets associés avec leur type, date de création et statut du jeton d'accès. Actions rapides : renouveler, supprimer, générer un jeton.

Gestion secrets

Actions



Formulaire d'ajout

Interface adaptative avec onglets selon le type de secret choisi (générique / certificat / RSA). Supporte l'upload de fichiers ou la génération automatique.

Onglets dynamiques

Upload/Génération



Interface Responsive

Bootstrap 5 + HTMX



Mon compte

Espace personnel pour la modification des informations, l'upload de photo de profil et le changement sécurisé de mot de passe.

Profil

Sécurité

API REST pour applications tierces

Objectif

Permettre à une application tierce de récupérer ses secrets de manière programmatique, sans jamais accéder à l'interface web, pour une intégration CI/CD fluide.

Mesures de Sécurité



Authentification Bearer

Token 256 bits généré cryptographiquement (secrets.token_hex).



Isolation Stricte

Un jeton = Un secret unique. Aucun accès croisé possible.



Audit & Notification

Email au propriétaire à chaque accès avec l'IP source.



Confidentialité

Aucune donnée sensible loggée côté serveur.

SDKS & EXEMPLES DISPONIBLES

 cURL

 Python

 JavaScript

 Node.js

 PHP

```
GET /api/v1/secrets/<nom_secret>/
Host: barbilink.internal
Authorization: Bearer <jeton_64_chars>
Accept: application/json
```



```
{
  "nom": "stripe-api-key",
  "type": "generic",
  "valeur": "sk_live_51Hx9zA...", // Le secret déchiffré
  "encodage": "text",
  "expiration": "2026-06-15T12:00:00Z"
}
```

Flux complet : du stockage à l'utilisation

1 Développeur

- ➔ Connexion à l'interface BarbiLink
- 📁 Création app `MonApp` & ajout secret
- 🔑 Génération du jeton d'accès

ACTION CRITIQUE : COPIE

📄 a3f8d2c1b4e5...

Clipboard

2 DevOps

- ⚙️ Configuration de l'environnement
- 🔒 Stockage du jeton dans la variable d'environnement `BARBILINK_TOKEN`

CONFIGURATION

```
# Variable d'environnement  
BARBILINK_TOKEN=a3f8d2...
```

.env / Shell

3 Production

- ⚡ Démarrage de l'application
- 🔌 Appel API avec le token

FLUX D'EXÉCUTION

REQ:

```
GET .../secrets/stripe-api-key/  
Auth: Bearer a3f8d2...
```

HTTP Exchange

RES:

```
{ "valeur": "sk_live..." }
```



Résultat : Le secret est utilisé sans jamais être écrit dans le code source.



Notifications et traçabilité

ACTION	NOTIFICATION ENVOYÉE
 Inscription	Email de bienvenue
 Création d'application	Confirmation avec nom de l'application
 Modification d'application	Alerte de sécurité avec ancien et nouveau nom
 Suppression d'application	Alerte avec liste des secrets supprimés Irréversiblement
 Suppression en masse	Alerte globale avec liste de toutes les applications et secrets supprimés
 Ajout d'un secret	Confirmation de stockage sécurisé dans le coffre-fort Barbican
 Renouvellement d'un secret	Confirmation avec le mode utilisé (manuel ou automatique)
 Génération d'un jeton	Rappel de sécurité : ne partager qu'avec des tiers de confiance
 Renouvellement d'un jeton	Alerte d'invalidation immédiate de l'ancien jeton d'accès
 Accès API (Tierce)	Alerte de traçabilité avec IP source et nom du secret consulté



Défis techniques rencontrés



BACKEND / AUTH

Intégration Keystone + Barbican

Chaque requête vers Barbican nécessite un token Keystone scopé sur le bon projet. La gestion du cycle de vie des tokens (obtention, scope spécifique, expiration) a nécessité le développement d'une couche d'abstraction robuste pour masquer cette complexité au reste de l'application.



SÉCURITÉ

Isolation forte des utilisateurs

Garantir qu'un utilisateur ne peut jamais accéder aux secrets d'un autre. Cela implique des contrôles d'accès stricts à chaque niveau, y compris sur les endpoints HTMX qui reçoivent des IDs en paramètres POST, afin de prévenir toute attaque de type IDOR (Insecure Direct Object Reference).



LOGIQUE MÉTIER

Renouvellement avec archivage

Le défi consistait à assurer la continuité de service lors du renouvellement. L'ancienne version du secret doit être conservée dans Barbican (archivage) pour permettre un rollback éventuel, tandis que la nouvelle version est activée instantanément, sans interruption pour les applications tierces consommatrices.



FRONTEND / UX

Interface réactive sans rechargement

L'utilisation d'HTMX pour charger dynamiquement les partiels de l'interface (création d'app, détails) a nécessité une gestion fine. Le défi principal était la gestion des scripts JavaScript injectés et la résolution des conflits de noms de fonctions entre les différents fragments HTML chargés asynchrones.

Fonctionnalités prévues

Système BarbiLink



Utilisateur

Développeur / Admin



App Tierce

Consommateur API



Orchestrateur

Processus interne

COMPTE

1. Gérer son compte

AUTH

9. S'authentifier (Keystone)

APPS

3. Gérer ses applications

SECRET

4. Ajouter un secret

SECRET

5. Renouveler un secret

SECRET

6. Révoquer un ancien secret

AUTO

2. Configurer renouvellement auto

API

7. Récupérer un secret

SYSTÈME

8. Notifier (Email)

SERVICES EXTERNES



Keystone

Identité & Auth



Barbican

Stockage Secrets



Service Mail

SMTP Gmail



ACME / CA

Certificats SSL



Gestion de compte



Applications



Secrets



Automatisation/API



Fonctionnalités réalisées : bilan



Entièrement Réalisées

Status: Opérationnel en Production

MAJORITAIRE



Comptes Utilisateurs

Création, modification, connexion/déconnexion, suppression. **Sécurité:** Logout auto après 3 min d'inactivité.



Applications & Isolation

Liste paginée, CRUD complet. **Backend:** Chaque app = 1 projet Keystone Isolé.



Gestion des Secrets

Ajout (Générique, SSL/TLS, RSA), Renouvellement manuel avec archivage des anciennes versions.



API & Intégration



Partiellement Réalisées

- **Renouvellement Automatique**
Moteur (daemon thread + commande) ✓ OK. Interface de configuration Admin ✗ Manquante.
- **ACME / Let's Encrypt**
Librairies intégrées. Flux complet d'obtention de certificat non finalisé.



Fonctionnalités Bonus



Recherche HTMX

Filtrage temps réel



Dashboard

Graphiques & Stats





Comment avons-nous atteint les objectifs ?



Intégration & Sécurité

BARBICAN & KEYSTONE



Stockage sécurisé Barbican

Utilisation du client officiel **python-barbicanclient** avec une couche d'abstraction. La base de données locale ne stocke que les **références (URLs)**, jamais les secrets en clair.



Isolation forte Keystone

Chaque application correspond à un **projet Keystone dédié**. Les tokens utilisés sont "scopés", garantissant mathématiquement qu'un utilisateur ne peut accéder qu'à ses propres ressources.



Automatisation & Accès

CYCLE DE VIE & API



Renouvellement automatique

Un **thread daemon** en arrière-plan interroge la base de données chaque jour. Il génère une nouvelle version du secret, archive l'ancienne et notifie le propriétaire par email.



Accès API pour applications tierces

Implémentation d'un modèle **TokenAcces** (Bearer 256 bits). L'API valide le jeton, récupère le payload via un compte admin et le délivre sans stockage intermédiaire.



Comment avons-nous atteint les objectifs ?



Intégration Barbican

python-barbicanclient

- ✓ Processus utilisant le client officiel avec une couche d'abstraction personnalisée.
- ✓ **Sécurité** : La base de données ne stocke jamais les secrets, uniquement leurs références (URLs).



Renouvellement Auto

Thread Daemon

- ✓ Processus d'arrière-plan (via apps.py) scannant les expirations chaque jour.
- ✓ **Archivage** : Conservation de l'ancienne référence (ref_old) lors de l'activation de la nouvelle.



Isolation Keystone

OS-Federation

- ✓ Création automatique d'un **projet dédié** pour chaque application.
- ✓ Usage de tokens scopés : un accès impossible aux secrets d'un autre projet.
- ✓ Révocation immédiate des credentials à la suppression de l'app.



API pour Apps Tierces

REST Framework

- ✓ Modèle TokenAcces unique (OneToOne) pour validation.
- ✓ Endpoint public validant le Bearer Token.
- ✓ Lecture du secret via un token admin temporaire et réponse JSON normalisée.



Conclusion

Réponse à la problématique initiale

BarbiLink élimine les risques liés à la gestion manuelle des secrets.



Les secrets ne transitent plus par le code source.



Chaque secret est chiffré et isolé dans Barbican sous un projet Keystone dédié.



Les applications tierces récupèrent leurs secrets à la demande via une API sécurisée.



Chaque accès est traçable et notifié au propriétaire en temps réel.



Le renouvellement des secrets est simple et sans interruption de service.

★ MISSION ACCOMPLIE



Démonstration Live

La plateforme est déployée et fonctionnelle. Testez la création de secrets et l'intégration API dès maintenant.

<http://192.168.1.180:8000>

100%
OPEN SOURCE

AES
CHIFFREMENT

REST
API STANDARD

wifi : Ng
password : nvf_20034